Chapter 1
Event and GUI Programming

THE APPLET CLASS

This chapter examines the **Applet** class, which provides the foundation for applets. The **Applet** class is contained in the **java.applet** package. **Applet** contains several methods that give you detailed control over the execution of your applet.

TWO TYPES OF APPLETS

It is important to state at the outset that there are two varieties of applets. The first are those based directly on the **Applet** class. These applets use the Abstract Window Toolkit (AWT) to provide the graphic user interface. This style of applet has been available since Java was first created. The second type of applets is those based on the Swing class **JApplet**. Swing applets use the Swing classes to provide the GUI. Swing offers a richer and often easier-to-use user interface than does the AWT. Thus, Swing-based applets are now the most popular.

APPLET BASICS

All applets are subclasses (either directly or indirectly) of **Applet**. Applets are not stand-alone programs. Instead, they run within either a web browser or an applet viewer. The illustrations shown in this chapter were created with the standard applet viewer, called **appletviewer**, provided by the JDK. Execution of an applet does not begin at **main( )**. Actually, few applets even have **main( )** methods. In non-Swing applets, output is handled with various AWT methods, such as **drawString( )**, which outputs a string to a specified X,Y location. Input is also handled differently than in a console application

To use an applet, it is specified in an HTMLfile. One way to do this is by using the APPLET tag. The applet will be executed by a Java-enabled web browser when it encounters the APPLET tag within the HTMLfile. To view and test an applet more conveniently, simply include a comment at the head of your Java source code file that contains the APPLET tag. This way, your code is documented with the necessary HTML statements needed by your applet, and you can test the compiled applet by starting the applet viewer with your Java source code file specified as the target. Here is an example of such a comment:

```
/*
<applet code="MyApplet" width=200 height=60>
</applet>
*/
```

This comment contains an APPLET tag that will run an applet called **MyApplet** in a window that is 200 pixels wide and 60 pixels high. Because the inclusion of an APPLET command makes testing applets easier

The Applet Class

The **Applet** class defines the methods shown in below table. **Applet** provides all necessary support for applet execution, such as starting and stopping. It also provides methods that load and display images, and methods that load and play audio clips. **Applet** extends the AWT class **Panel**. In turn, **Panel** extends **Container**, which extends **Component**. These classes provide support for Java's window-based, graphical interface. Thus, **Applet** provides all of the necessary support for window-based activities.

Method	Description
void destroy( )	Called by the browser just before an applet is terminated. Your applet will override this method if it needs to perform any cleanup prior to its destruction.
AccessibleContext getAccessibleContext( )	Returns the accessibility context for the invoking object.
AppletContext getAppletContext( )	Returns the context associated with the applet.
String getAppletInfo( )	Returns a string that describes the applet.
AudioClip getAudioClip(URL *url*)	Returns an **AudioClip** object that encapsulates the audio clip found at the location specified by *url*.
AudioClip getAudioClip(URL *url*, String *clipName*)	Returns an **AudioClip** object that encapsulates the audio clip found at the location specified by *url* and having the name specified by *clipName*.
URL getCodeBase( )	Returns the URL associated with the invoking applet.
URL getDocumentBase( )	Returns the URL of the HTML document that invokes the applet.
Image getImage(URL *url*)	Returns an **Image** object that encapsulates the image found at the location specified by *url*.
Image getImage(URL *url*, String *imageName*)	Returns an **Image** object that encapsulates the image found at the location specified by *url* and having the name specified by *imageName*.
Locale getLocale( )	Returns a **Locale** object that is used by various locale-sensitive classes and methods.
String getParameter(String *paramName*)	Returns the parameter associated with *paramName*. **null** is returned if the specified parameter is not found.
String[ ] [ ] getParameterInfo( )	Returns a **String** table that describes the parameters recognized by the applet. Each entry in the table must consist of three strings that contain the name of the parameter, a description of its type and/or range, and an explanation of its purpose.
void init( )	Called when an applet begins execution. It is the first method called for any applet.

boolean isActive( )	Returns **true** if the applet has been started. It returns **false** if the applet has been stopped.
static final AudioClip newAudioClip(URL *url*)	Returns an **AudioClip** object that encapsulates the audio clip found at the location specified by *url*. This method is similar to **getAudioClip( )** except that it is static and can be executed without the need for an **Applet** object.
void play(URL *url*)	If an audio clip is found at the location specified by *url*, the clip is played.
void play(URL *url*, String *clipName*)	If an audio clip is found at the location specified by *url* with the name specified by *clipName*, the clip is played.
void resize(Dimension *dim*)	Resizes the applet according to the dimensions specified by *dim*. **Dimension** is a class stored inside **java.awt**. It contains two integer fields: **width** and **height**.
void resize(int *width*, int *height*)	Resizes the applet according to the dimensions specified by *width* and *height*.
final void setStub(AppletStub *stubObj*)	Makes *stubObj* the stub for the applet. This method is used by the run-time system and is not usually called by your applet. A *stub* is a small piece of code that provides the linkage between your applet and the browser.

APPLET ARCHITECTURE

An applet is a window-based program. Its architecture is different from the console-based programs.

First, applets are event driven. An applet waits until an event occurs. The run-time system notifies the applet about an event by calling an event handler that has been provided by the applet. Once this happens, the applet must take appropriate action and then quickly return. It must perform specific actions in response to events and then return control to the run-time system.

Second, the user initiates interaction with an applet. As you know, in a non-windowed program, when the program needs input, it will prompt the user and then call some input method, such as **readLine( )**. This is not the way it works in an applet. Instead, the user interacts with the applet as he or she wants, when he or she wants. These interactions are sent to the applet as events to which the applet must respond. For example, when the user clicks the mouse inside the applet's window, a mouse-clicked event is generated. If the user presses a key while the applet's window has input focus, a key press event is generated. Applets can contain various controls, such as push buttons and check boxes. When the user interacts with one of these controls, an event is generated.

While the architecture of an applet is not as easy to understand as that of a console-based program, Java makes it as simple as possible.

AN APPLET SKELETON

An applet has four methods, **init( )**, **start( )**, **stop( )**, and **destroy( )**. These apply to all applets and are defined by **Applet**. AWT-based applets will also override the **paint( )** method, which is defined by the AWT **Component** class. This method is called when the applet's output must be redisplayed. These five methods can be assembled into the skeleton shown here:

```
// An Applet skeleton.
import java.awt.*;
import java.applet.*;
/*
<applet code="AppletSkel" width=300 height=100>
</applet>
*/
public class AppletSkel extends Applet {
// Called first.
public void init() {
// initialization
}
/* Called second, after init(). Also called whenever
the applet is restarted. */
public void start() {
// start or resume execution
}
// Called when the applet is stopped.
public void stop() {
// suspends execution
}
/* Called when applet is terminated. This is the last
method executed. */
public void destroy() {
// perform shutdown activities
}
// Called when an applet's window must be restored.
public void paint(Graphics g) {
// redisplay contents of window
}
}
```

Although this skeleton does not do anything, it can be compiled and run. When run, it generates the following window when viewed with an applet viewer:

Applet Initialization and Termination

It is important to understand the order in which the various methods shown in the skeleton are called. When an applet begins, the following methods are called, in this sequence:

1. **init( )**
2. **start( )**
3. **paint( )**

When an applet is terminated, the following sequence of method calls takes place:

1. **stop( )**
2. **destroy( )**

init( )

The **init( )** method is the first method to be called. This is where you should initialize variables. This method is called only once during the run time of your applet.

start( )

The **start( )** method is called after **init( )**. It is also called to restart an applet after it has been stopped. Whereas **init( )** is called once—the first time an applet is loaded—**start( )** is called each time an applet's HTML document is displayed onscreen. So, if a user leaves a web page and comes back, the applet resumes execution at **start( )**.

paint( )

The **paint( )** method is called each time your applet's output must be redrawn. This situation can occur for several reasons. For example, the window in which the applet is running may be overwritten by another window and then uncovered. Or the applet window may be minimized and then restored. **paint( )** is also called when the applet begins execution. Whatever the cause, whenever the applet must redraw its output, **paint( )** is called. The **paint( )** method has one parameter of type **Graphics**. This parameter will contain the graphics context, which describes the graphics environment in which the applet is running.

stop( )

The **stop( )** method is called when a web browser leaves the HTML document containing the applet—when it goes to another page, for example. When **stop( )** is called, the applet is probably running. You should use **stop( )** to suspend threads

that don't need to run when the applet is not visible. You can restart them when **start( )** is called if the user returns to the page.

destroy( )

The **destroy( )** method is called when the environment determines that your applet needs to be removed completely from memory. At this point, you should free up any resources the applet may be using. The **stop( )** method is always called before **destroy( )**.

Overriding update( )

In some situations, your applet may need to override another method defined by the AWT, called **update( )**. This method is called when your applet has requested that a portion of its window be redrawn. The default version of **update( )** simply calls **paint( )**.

SIMPLE APPLET DISPLAY METHODS

To output a string to an applet, use **drawString( )**, which is a member of the **Graphics** class. Typically, it is called from within either **update( )** or **paint( )**. It has the following general form:

```
void drawString(String message, int x, int y)
```

Here, *message* is the string to be output beginning at *x,y*. In a Java window, the upper-left corner is location 0,0. The **drawString( )** method will not recognize newline characters. If you want to start a line of text on another line, you must do so manually, specifying the precise X,Y location where you want the line to begin.

To set the background color of an applet's window, use **setBackground( )**. To set the foreground color, use **setForeground( )**. They have the following general forms:

```
void setBackground(Color newColor)
void setForeground(Color newColor)
```

Here, *newColor* specifies the new color. The class **Color** defines the constants shown here that can be used to specify colors:

Color.black	Color.magenta	Color.blue	Color.orange
Color.cyan	Color.pink	Color.darkGray	Color.red
Color.gray	Color.white	Color.green	Color.yellow
Color.lightGray			

Uppercase versions of the constants are also defined.

The following example sets the background color to green and the text color to red:

```
setBackground(Color.green);
setForeground(Color.red);
```

A good place to set the foreground and background colors is in the **init( )** method. You can change these colors as often as necessary during the execution of your applet. You can obtain the current settings for the background and foreground

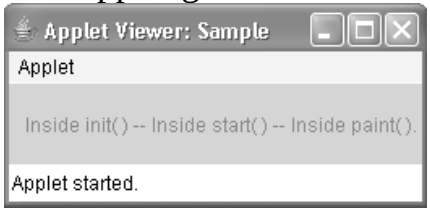
colors by calling **getBackground( )** and **getForeground( )**, respectively. They are also defined by **Component** and are shown here:

```
Color getBackground( )
Color getForeground( )
```

Here is a very simple applet that sets the background color to cyan, the foreground color to red, and displays a message that illustrates the order in which the **init( )**, **start( )**, and **paint( )** methods are called when an applet starts up:

```
/* A simple applet that sets the foreground and
background colors and outputs a string. */
import java.awt.*;
import java.applet.*;
/*
<applet code="Sample" width=300 height=50>
</applet>
*/
public class Sample extends Applet{
String msg;
// set the foreground and background colors.
public void init() {
setBackground(Color.cyan);
setForeground(Color.red);
msg = "Inside init( ) --";
}
// Initialize the string to be displayed.
public void start() {
msg += " Inside start( ) --";
}
// Display msg in applet window.
public void paint(Graphics g) {
msg += " Inside paint( ).";
g.drawString(msg, 10, 30);
}
}
```

This applet generates the window shown here:



REQUESTING REPAINTING

As a general rule, an applet writes to its window only when its **update( )** or **paint()** method is called by the AWT. This raises an interesting question: How can the applet itself cause its window to be updated when its information changes? For example, if an applet is displaying a moving banner, what mechanism does the applet use to update the window each time this banner scrolls? An applet must quickly return control to the run-time system. It cannot create a loop inside **paint( )** that repeatedly scrolls the banner.

Whenever your applet needs to update the information displayed in its window, it simply calls **repaint( )**. The **repaint( )** method is defined by the AWT. It causes the AWT run-time system to execute a call to your applet's **update( )** method, which, calls **paint( )**.

The simplest version of **repaint( )** is shown here:

```
void repaint( )
```

This version causes the entire window to be repainted. The following version specifies a region that will be repainted:

```
void repaint(int left, int top, int width, int height)
```

Here, the coordinates of the upper-left corner of the region are specified by *left* and *top,* and the width and height of the region are passed in *width* and *height.* These dimensions are specified in pixels. You save time by specifying a region to repaint.

THE HTML APPLET TAG

The APPLET tag be used to start an applet from both an HTML document and from an applet viewer. An applet viewer will execute each APPLET tag. So far, we have been using only a simplified form of the APPLET tag. The syntax for a fuller form of the APPLET tag is shown here. Bracketed items are optional.

```
< APPLET
[CODEBASE = codebaseURL]
CODE = appletFile
[ALT = alternateText]
[NAME = appletInstanceName]
WIDTH = pixels HEIGHT = pixels
[ALIGN = alignment]
[VSPACE = pixels] [HSPACE = pixels]
>
[< PARAM NAME = AttributeName VALUE = AttributeValue>]
[< PARAM NAME = AttributeName2 VALUE = AttributeValue>]
. . .
[HTML Displayed in the absence of Java]
</APPLET>
```

CODEBASE

CODEBASE is an optional attribute that specifies the base URL of the applet Code.

CODE

CODE is a required attribute that gives the name of the file containing your applet's compiled .class file.

ALT

The ALT tag is an optional attribute used to specify a short text message that should be displayed

NAME

NAME is an optional attribute used to specify a name for the applet instance. Applets must be named in order for other applets on the same page to find them by name and communicate with them.

WIDTH and HEIGHT

WIDTH and HEIGHT are required attributes that give the size (in pixels) of the applet display area.

ALIGN

ALIGN is an optional attribute that specifies the alignment of the applet. The possible values are: LEFT, RIGHT, TOP, BOTTOM, MIDDLE, BASELINE, TEXTTOP, ABSMIDDLE, and ABSBOTTOM.

VSPACE and HSPACE

These attributes are optional. VSPACE specifies the space, in pixels, above and below the applet. HSPACE specifies the space, in pixels, on each side of the applet.

PARAM NAME and VALUE

The PARAM tag allows you to specify applet-specific arguments in an HTML page. Applets access their attributes with the getParameter( ) method.

Chapter 2
Event Handling

Applets are event-driven programs. Thus, event handling is at the core of successful applet programming. Most events to which your applet will respond are generated by the user. These events are passed to your applet in a variety of ways, with the specific method depending upon the actual event. There are several types of events. The most commonly handled events are those generated by the mouse, the keyboard, and various controls, such as a push button. Events are supported by the java.awt.event package.

THE DELEGATION EVENT MODEL

The Java approach to handling events is based on the delegation event model. The delegation event model defines standard and consistent mechanisms to generate

and process events. Its concept is quite simple: a source generates an event and sends it to one or more listeners. In this scheme, the listener simply waits until it receives an event. Once received, the listener processes the event and then returns. The advantage of this design is that the logic that processes events is cleanly separated from the user interface logic that generates those events. A user interface element is able to "delegate" the processing of an event to a separate piece of code. In the delegation event model, listeners must register with a source in order to receive an event notification.

Events

In the delegation model, an event is an object that describes a state change in a source. It can be generated as a consequence of a person interacting with the elements in a graphical user interface, such as pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse.

Event Sources

An event source is an object that generates an event. A source must register listeners in order for the listener to receive notifications about a specific type of event. Each type of event has its own registration method. Here is the general form:

```
public void addTypeListener(TypeListenerel)
```

Here, Type is the name of the event, and el is a reference to the event listener. For example, the method that registers a keyboard event listener is called addKeyListener( ). The method that registers a mouse motion listener is called addMouseMotionListener( ). When an event occurs, all registered listeners are notified and receive a copy of the event object.

A source must also provide a method that allows a listener to unregister an interest in a specific type of event. The general form of such a method is this:

```
public void removeTypeListener(TypeListenerel)
```

Here, Type is the name of the event, and el is a reference to the event listener. For example, to remove a keyboard listener, you would call removeKeyListener( ).

Event Listeners

A listener is an object that is notified when an event occurs. It has two major requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications. The methods that receive and process events are defined in a set of interfaces found injava.awt.event. For example, the MouseMotionListener interface defines methods that receive notifications when the mouse is dragged or moved. Any object may receive and process one or both of these events if it provides an implementation of this interface.

EVENT CLASSES

The classes that represent events are at the core of Java's event handling mechanism. At the root of the Java event class hierarchy is EventObject, which is in java.util. It is the super class for all events. The class AWTEvent, defined within the java.awt package, is a subclass of EventObject. It is the superclass (either directly or indirectly) for all AWT-based events used by the delegation event model.

The package java.awt.eventdefines several types of events that are generated by varioususer interface elements. Table below give the most commonly used ones and provides a brief description of when they are generated.

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged or moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table 1 The Main Event Classes in **java.awt.event**

The ActionEvent Class

An **ActionEvent** is generated when a button is pressed, a list item is double-clicked, or a menu item is selected. The **ActionEvent** class defines an integer constant, **ACTION_PERFORMED**, which can be used to identify action events. **ActionEvent** has these three constructors:

```
ActionEvent(Object src, int type, String cmd)
ActionEvent(Object src, int type, String cmd, int modifiers)
ActionEvent(Object src, int type, String cmd, long when, int modifiers)
```

Here, *src* is a reference to the object that generated this event. The type of the event is specified by *type*, and its command string is *cmd*. The argument *modifiers* indicates which modifier keys were pressed when the event was generated. The *when* parameter specifies when the event occurred.

The KeyEvent Class

A **KeyEvent** is generated when keyboard input occurs. There are three types of key events, which are identified by these integer constants: **KEY_PRESSED**,

KEY_RELEASED, and **KEY_TYPED**. The first two events are generated when any key is pressed or released. The last event occurs only when a character is generated. Remember, not all keypresses result in characters. For example, pressing SHIFT does not generate a character.

KeyEvent is a subclass of **InputEvent**. Here is one of its constructors:

```
KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)
```

The MouseEvent Class

There are eight types of mouse events. The **MouseEvent** class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

EVENT LISTENER INTERFACES

Event listeners receive event notifications. Listeners are created by implementing one or more of the interfaces defined by the java.awt.event package. When an event occurs, the event source invokes the appropriate method defined by the listener and provides an event object as its argument. Table below lists commonly used listener interfaces and provides a brief description of the methods they define.

Interface	Description
ActionListener	Defines one method to receive action events. Action events are generated by such things as push buttons and menus.
AdjustmentListener	Defines one method to receive adjustment events, such as those produced by a scroll bar.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes. An item event is generated by a check box, for example.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.

The ActionListener Interface

This interface defines the **actionPerformed( )** method that is invoked when an action event occurs. Its general form is shown here:

```
void actionPerformed(ActionEvent ae)
```

The KeyListener Interface

This interface defines three methods. The **keyPressed( )** and **keyReleased( )** methods are invoked when a key is pressed and released, respectively. The **keyTyped( )** method is invoked when a character has been entered.

For example, if a user presses and releases the A key, three events are generated in sequence: key pressed, typed, and released. If a user presses and releases the HOME key, two key events are generated in sequence: key pressed and released.

The general forms of these methods are shown here:

```
void keyPressed(KeyEvent ke)
void keyReleased(KeyEvent ke)
void keyTyped(KeyEvent ke)
```

The MouseListener Interface

This interface defines five methods. If the mouse is pressed and released at the same point, **mouseClicked( )** is invoked. When the mouse enters a component, the **mouseEntered( )** method is called. When it leaves, **mouseExited( )** is called. The **mousePressed( )** and **mouseReleased( )** methods are invoked when the mouse is pressed and released, respectively.

The general forms of these methods are shown here:

```
void mouseClicked(MouseEvent me)
void mouseEntered(MouseEvent me)
void mouseExited(MouseEvent me)
void mousePressed(MouseEvent me)
void mouseReleased(MouseEvent me)
```

USING THE DELEGATION EVENT MODEL

To use delegation event model follow these two steps:

1. Implement the appropriate interface in the listener so that it will receive the type of event desired.
2. Implement code to register and unregister (if necessary) the listener as a recipient for the event notifications.

Remember that a source may generate several types of events. Each event must be registered separately. Also, an object may register to receive several types of events, but it must implement all of the interfaces that are required to receive these events.

Handling Mouse Events

To handle mouse events, you must implement the **MouseListener** and the **MouseMotionListener** interfaces.

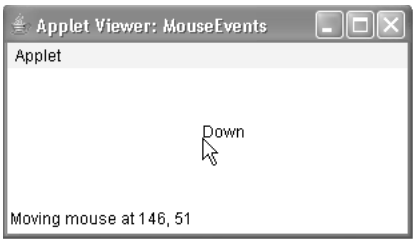
The following applet demonstrates the process. It displays the current coordinates of the mouse in the applet's status window. Each time a button is pressed, the word "Down" is displayed at the location of the mouse pointer. Each time the button is

released, the word “Up” is shown. If a button is clicked, the message “Mouse clicked” is displayed in the upperleft corner of the applet display area. As the mouse enters or exits the applet window, a message is displayed in the upper-left corner of the applet display area. When dragging the mouse, a * is shown, which tracks with the mouse pointer as it is dragged. Notice that the two variables, **mouseX** and **mouseY**, store the location of the mouse when a mouse pressed, released, or dragged event occurs. These coordinates are then used by **paint( )** to display output at the point of these occurrences.

```
// Demonstrate the mouse event handlers.
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="MouseEvents" width=300 height=100>
</applet>
*/
public class MouseEvents extends Applet
implements MouseListener, MouseMotionListener {
String msg = "";
int mouseX = 0, mouseY = 0; // coordinates of mouse
public void init() {
addMouseListener(this);
addMouseMotionListener(this);
}
// Handle mouse clicked.
public void mouseClicked(MouseEvent me) {
// save coordinates
mouseX = 0;
mouseY = 10;
msg = "Mouse clicked.";
repaint();
}
// Handle mouse entered.
public void mouseEntered(MouseEvent me) {
// save coordinates
mouseX = 0;
mouseY = 10;
msg = "Mouse entered.";
repaint();
}
// Handle mouse exited.
public void mouseExited(MouseEvent me) {
// save coordinates
mouseX = 0;
```

```
mouseY = 10;
msg = "Mouse exited.";
repaint();
}
// Handle button pressed.
public void mousePressed(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "Down";
repaint();
}
// Handle button released.
public void mouseReleased(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "Up";
repaint();
}
// Handle mouse dragged.
public void mouseDragged(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "*";
showStatus("Dragging mouse at " + mouseX + ", " + mouseY);
repaint();
}
// Handle mouse moved.
public void mouseMoved(MouseEvent me) {
// show status
showStatus("Moving mouse at " + me.getX() + ", " + me.getY());
}
// Display msg in applet window at current X,Y location.
public void paint(Graphics g) {
g.drawString(msg, mouseX, mouseY);
}
}
```

Sample output from this program is shown here:



Let's look closely at this example. The **MouseEvents** class extends **Applet** and implements both the **MouseListener** and **MouseMotionListener** interfaces. These two interfaces contain methods that receive and process the various types of mouse events.

Inside **init( )**, the applet registers itself as a listener for mouse events. This is done by using **addMouseListener( )** and **addMouseMotionListener( )**. They are shown here:

```
void addMouseListener(MouseListener ml)
void addMouseMotionListener(MouseMotionListener mml)
```

Here, *ml* is a reference to the object receiving mouse events, and *mml* is a reference to the object receiving mouse motion events. In this program, the same object is used for both.

The applet then implements all of the methods defined by the **MouseListener** and **MouseMotionListener** interfaces. These are the event handlers for the various mouse events. Each method handles its event and then returns.

Handling Keyboard Events

To handle keyboard events, you use the same general architecture as that shown in the mouse event example in the preceding section. The difference is that you will be implementing the **KeyListener** interface. Before looking at an example, it is useful to review how key events are generated. When a key is pressed, a **KEY_PRESSED** event is generated. This results in a call to the **keyPressed( )** event handler. When the key is released, a **KEY_RELEASED** event is generated and the **keyReleased( )** handler is executed. If a character is generated by the keystroke, then a **KEY_TYPED** event is sent and the **keyTyped( )** handler is invoked. Thus, each time the user presses a key, at least two and often three events are generated. If your program needs to handle special keys, such as the arrow or function keys, then it must watch for them through the **keyPressed( )** handler.

The following program demonstrates keyboard input. It echoes keystrokes to the applet window and shows the pressed/released status of each key in the status window.

```
// Demonstrate the key event handlers.
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
```

```
<applet code="SimpleKey" width=300 height=100>
</applet>
*/
public class SimpleKey extends Applet
implements KeyListener {
String msg = "";
int X = 10, Y = 20; // output coordinates
public void init() {
addKeyListener(this);
}
public void keyPressed(KeyEvent ke) {
showStatus("Key Down");
}
public void keyReleased(KeyEvent ke) {
showStatus("Key Up");
}
public void keyTyped(KeyEvent ke) {
msg += ke.getKeyChar();
repaint();
}
// Display keystrokes.
public void paint(Graphics g) {
g.drawString(msg, X, Y);
}
}
```

Sample output is shown here:

Chapter 3
Window fundamentals

WINDOW FUNDAMENTALS

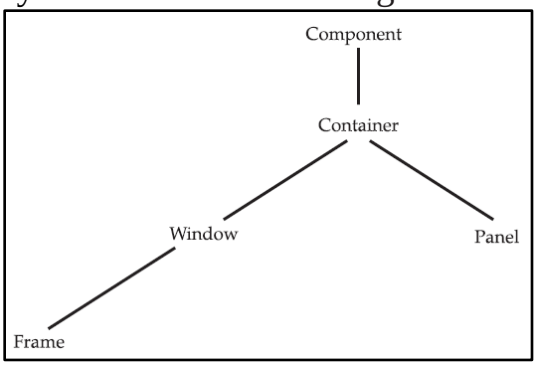
The AWT defines windows according to a class hierarchy that adds functionality and specificity with each level. The two most common windows are those derived from **Panel**, which is used by applets, and those derived from **Frame**, which creates a standard application window. Much of the functionality of these windows is derived from their parent classes. The below figure shows the class hierarchy for **Panel** and **Frame**. Let's look at each of these classes now.

Component

At the top of the AWT hierarchy is the **Component** class. **Component** is an abstract class that encapsulates all of the attributes of a visual component. All user interface elements that are displayed on the screen and that interact with the user are subclasses of **Component**. It defines over a hundred public methods that are responsible for managing events, such as mouse and keyboard input, positioning and sizing the window, and repainting. A **Component** object is responsible for remembering the current foreground and background colors and the currently selected text font.

Container

The **Container** class is a subclass of **Component**. It has additional methods that allow other **Component** objects to be nested within it. Other **Container** objects can be stored inside of a **Container**. This makes for a multileveled containment system. It does this through the use of various layout managers



Panel

The **Panel** class is a concrete subclass of **Container**. It doesn't add any new methods; it simply implements **Container**. **Panel** is the superclass for **Applet**. A **Panel** is a window that does not contain a title bar, menu bar, or border. This is why you don't see these items when an applet is run inside a browser. When you run an applet using an applet viewer, the applet viewer provides the title and border. Other components can be added to a **Panel** object by its **add( )** method. Once these components have been added, you can position and resize them manually using the **setLocation( )**, **setSize( )**, **setPreferredSize( )**, or **setBounds( )** methods defined by **Component**.

Window

The **Window** class creates a top-level window. A*top-level window* is not contained within any other object; it sits directly on the desktop. Generally, you won't create **Window** objects directly. Instead, you will use a subclass of **Window** called **Frame**.

Frame

Frame encapsulates what is commonly thought of as a "window." It is a subclass of **Window** and has a title bar, menu bar, borders, and resizing corners. If you create a **Frame** object from within an applet, it will contain a warning message, such as "Java Applet Window," to the user that an applet window has been created. This message warns users that the window they see was started by an applet and not by software running on their computer. When a **Frame** window is created by a stand-alone application rather than an applet, a normal window is created.

Canvas

Canvas encapsulates a blank window upon which you can draw.

WORKING WITH FRAME WINDOWS

After the applet, the type of window you will most often create is derived from **Frame**. Here are two of **Frame**'s constructors:

```
Frame( )
Frame(String title)
```

The first form creates a standard window that does not contain a title. The second form creates a window with the title specified by *title.* Notice that you cannot specify the dimensions of the window. Instead, you must set the size of the window after it has been created.

There are several key methods you will use when working with **Frame** windows. They are examined here.

Setting the Window's Dimensions

The **setSize( )** method is used to set the dimensions of the window. Its signature is shown here:

```
void setSize(int newWidth, int newHeight)
void setSize(Dimension newSize)
```

The new size of the window is specified by *newWidth* and *newHeight*, or by the **width** and **height** fields of the **Dimension** object passed in *newSize*. The dimensions are specified in terms of pixels.

The **getSize( )** method is used to obtain the current size of a window. Its signature is shown here:

```
Dimension getSize( )
```

This method returns the current size of the window contained within the **width** and **height** fields of a **Dimension** object.

Hiding and Showing a Window

After a frame window has been created, it will not be visible until you call **setVisible( )**. Its signature is shown here:

```
void setVisible(boolean visibleFlag)
```

The component is visible if the argument to this method is **true**. Otherwise, it is hidden.

Setting a Window's Title

You can change the title in a frame window using **setTitle( )**, which has this general form:

```
void setTitle(String newTitle)
```

Here, *newTitle* is the new title for the window.

Closing a Frame Window

When using a frame window, your program must remove that window from the screen when it is closed, by calling **setVisible(false)**. To intercept a window-close event, you must implement
the **windowClosing( )** method of the **WindowListener** interface. Inside **windowClosing( )**, you must remove the window from the screen.

CREATING A FRAME WINDOW IN AN APPLET

Creating a new frame window from within an applet is actually quite easy. First, create a subclass of **Frame**. Next, override any of the standard applet methods, such as **init( )**, **start( )**, and **stop( )**, to show or hide the frame as needed. Finally, implement the **windowClosing( )** method of the **WindowListener** interface, calling **setVisible(false)** when the window is closed.

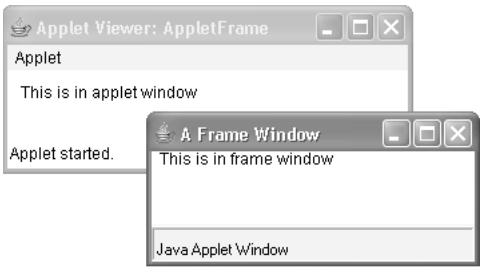
Once you have defined a **Frame** subclass, you can create an object of that class. This causes a frame window to come into existence, but it will not be initially visible. You make it visible by calling **setVisible( )**. When created, the window is given a default height and width. You can set the size of the window explicitly by calling the **setSize( )** method.

The following applet creates a subclass of **Frame** called **SampleFrame**. A window of this subclass is instantiated within the **init( )** method of **AppletFrame**. Notice that **SampleFrame** calls **Frame**'s constructor. This causes a standard frame window to be created with the title passed in **title**. This example overrides the applet's **start( )** and **stop( )** methods so that they show and hide the child window, respectively. This causes the window to be removed automatically when you terminate the applet, when you close the window, or, if using a browser, when you move to another page. It also causes the child window to be shown when the browser returns to the applet.

```
// Create a child frame window from within an applet.
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="AppletFrame" width=300 height=50>
</applet>
*/
// Create a subclass of Frame.
class SampleFrame extends Frame {
```

```
SampleFrame(String title) {
super(title);
// create an object to handle window events
MyWindowAdapter adapter = new MyWindowAdapter(this);
// register it to receive those events
addWindowListener(adapter);
}
public void paint(Graphics g) {
g.drawString("This is in frame window", 10, 40);
}
}
class MyWindowAdapter extends WindowAdapter {
SampleFrame sampleFrame;
public MyWindowAdapter(SampleFrame sampleFrame) {
this.sampleFrame = sampleFrame;
}
public void windowClosing(WindowEvent we) {
sampleFrame.setVisible(false);
}
}
// Create frame window.
public class AppletFrame extends Applet {
Frame f;
public void init() {
f = new SampleFrame("A Frame Window");
f.setSize(250, 250);
f.setVisible(true);
}
public void start() {
f.setVisible(true);
}
public void stop() {
f.setVisible(false);
}
public void paint(Graphics g) {
g.drawString("This is in applet window", 10, 20);
}
}
```

Sample output from this program is shown here:



HANDLING EVENTS IN A FRAME WINDOW

Since **Frame** is a subclass of **Component**, it inherits all the capabilities defined by **Component**. This means that you can use and manage a frame window just like you manage an applet's main window. For example, you can override **paint( )** to display output, call **repaint( )** when you need to restore the window, and add event handlers. Whenever an event occurs in a window, the event handlers defined by that window will be called. Each window handles its own events. For example, the following program creates a window that responds to mouse events. The main applet window also responds to mouse events. When you experiment with this program, you will see that mouse events are sent to the window in which the event occurs.

```
// Handle mouse events in both child and applet windows.
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="WindowEvents" width=300 height=50>
</applet>
*/
// Create a subclass of Frame.
class SampleFrame extends Frame
implements MouseListener, MouseMotionListener {
String msg = "";
int mouseX=10, mouseY=40;
int movX=0, movY=0;
SampleFrame(String title) {
super(title);
// register this object to receive its own mouse events
addMouseListener(this);
addMouseMotionListener(this);
// create an object to handle window events
MyWindowAdapter adapter = new MyWindowAdapter(this);
// register it to receive those events
addWindowListener(adapter);
}
// Handle mouse clicked.
public void mouseClicked(MouseEvent me) {
```

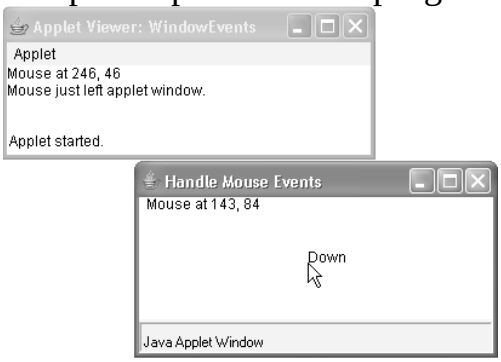
```
}
// Handle mouse entered.
public void mouseEntered(MouseEvent evtObj) {
// save coordinates
mouseX = 10;
mouseY = 54;
msg = "Mouse just entered child.";
repaint();
}
// Handle mouse exited.
public void mouseExited(MouseEvent evtObj) {
// save coordinates
mouseX = 10;
mouseY = 54;
msg = "Mouse just left child window.";
repaint();
}
// Handle mouse pressed.
public void mousePressed(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "Down";
repaint();
}
// Handle mouse released.
public void mouseReleased(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "Up";
repaint();
}
// Handle mouse dragged.
public void mouseDragged(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
movX = me.getX();
movY = me.getY();
msg = "*";
repaint();
}
// Handle mouse moved.
public void mouseMoved(MouseEvent me) {
// save coordinates
```

```
movX = me.getX();
movY = me.getY();
repaint(0, 0, 100, 60);
}
public void paint(Graphics g) {
g.drawString(msg, mouseX, mouseY);
g.drawString("Mouse at " + movX + ", " + movY, 10, 40);
}
}
class MyWindowAdapter extends WindowAdapter {
SampleFrame sampleFrame;
public MyWindowAdapter(SampleFrame sampleFrame) {
this.sampleFrame = sampleFrame;
}
public void windowClosing(WindowEvent we) {
sampleFrame.setVisible(false);
}
}
// Applet window.
public class WindowEvents extends Applet
implements MouseListener, MouseMotionListener {
SampleFrame f;
String msg = "";
int mouseX=0, mouseY=10;
int movX=0, movY=0;
// Create a frame window.
public void init() {
f = new SampleFrame("Handle Mouse Events");
f.setSize(300, 200);
f.setVisible(true);
// register this object to receive its own mouse events
addMouseListener(this);
addMouseMotionListener(this);
}
// Remove frame window when stopping applet.
public void stop() {
f.setVisible(false);
}
// Show frame window when starting applet.
public void start() {
f.setVisible(true);
}
// Handle mouse clicked.
public void mouseClicked(MouseEvent me) {
}
// Handle mouse entered.
```

```
public void mouseEntered(MouseEvent me) {
// save coordinates
mouseX = 0;
mouseY = 24;
msg = "Mouse just entered applet window.";
repaint();
}
// Handle mouse exited.
public void mouseExited(MouseEvent me) {
// save coordinates
mouseX = 0;
mouseY = 24;
msg = "Mouse just left applet window.";
repaint();
}
// Handle button pressed.
public void mousePressed(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "Down";
repaint();
}
// Handle button released.
public void mouseReleased(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "Up";
repaint();
}
// Handle mouse dragged.
public void mouseDragged(MouseEvent me) {
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
movX = me.getX();
movY = me.getY();
msg = "*";
repaint();
}
// Handle mouse moved.
public void mouseMoved(MouseEvent me) {
// save coordinates
movX = me.getX();
movY = me.getY();
```

```
repaint(0, 0, 100, 20);
}
// Display msg in applet window.
public void paint(Graphics g) {
g.drawString(msg, mouseX, mouseY);
g.drawString("Mouse at " + movX + ", " + movY, 0, 10);
}
}
```

Sample output from this program is shown here:



CREATING A WINDOWED PROGRAM

Although creating applets is a common use for Java's AWT, it is also possible to create stand-alone AWT-based applications. To do this, simply create an instance of the window or windows you need inside **main( )**.

Chapter 4
Introducing Swing

SWING FEATURES

• **Light Weight** - Swing component are independent of native Operating System's API as Swing API controls are rendered mostly using pure JAVA code instead of underlying operating system calls.

• **Rich controls** - Swing provides a rich set of advanced controls like Tree, TabbedPane, slider, colorpicker, table controls

• **Highly Customizable** - Swing controls can be customized in very easy way as visual apperance is independent of internal representation.

• **Pluggable look-and-feel-** SWING based GUI Application look and feel can be changed at run time based on available values.

COMPONENTS AND CONTAINERS

A Swing GUI consists of two key items: components and containers. However, this distinction is mostly conceptual because all containers are also components. The difference between the two is found in their intended purpose: As the term is commonly used, a component is an independent visual control, such as a push button or text field. A container holds a group of components. Thus, a container is a special type of component that is designed to hold other components.

Furthermore, in order for a component to be displayed, it must be held within a container. Thus, all Swing GUIs will have at least one container. Because containers are components, a container can also hold other containers. This enables Swing to define what is called a containment hierarchy, at the top of which must be a top level container.

Components

In general, Swing components are derived from the JComponent class. JComponent provides the functionality that is common to all components. For example, JComponent supports the pluggable look and feel. JComponent inherits the AWT classes Container and Component.

All of Swing's components are represented by classes defined within the package javax.swing. The following table shows the class names for Swing components. Notice that all component classes begin with the letter J. For example, the class for a label is JLabel, the class for a push button is JButton, and the class for a check box isJCheckBox.

Containers

Swing defines two types of containers. The first are top-level containers: JFrame, JApplet, JWindow, and JDialog. These containers do not inherit JComponent. They do, however, inherit the AWT classes Component and Container. The top-level containers are heavyweight. As the name implies, a top-level container must be at the top of a containment hierarchy. The one most commonly used for applications is **JFrame**. The one used for applets is **JApplet**.

The second types of container supported by Swing are lightweight containers. Lightweight containers *do* inherit **JComponent**. An example of a lightweight container is **JPanel**, which is a general-purpose container. Lightweight containers are often used to organize and manage groups of related components.

The Top-Level Container Panes

Each top-level container defines a set of panes. At the top of the hierarchy is an instance of JRootPane. JRootPane is a lightweight container whose purpose is to manage the other panes. It also helps manage the optional menu bar. The panes that compose the **root pane are** called **the glass pane, the content pane, and the layered pane**.

The glass pane is the top-level pane. It sits above and completely covers all other panes. The glass pane enables you to manage mouse events. The layered pane allows components to be given a depth value. This value determines which component overlays another. The layered pane holds the content pane and the (optional) menu bar. The pane with which your application will interact the most is the content pane, because this is the pane to which you will add visual components. Therefore, the content pane holds the components that the user interacts with.

THE SWING PACKAGES

Swing is a very large subsystem and makes use of many packages.

javax.swing	javax.swing.border	javax.swing.colorchooser	javax.swing.event
javax.swing.filechooser	javax.swing.plaf.basic	javax.swing.plaf.metal	javax.swing.plaf.multi
javax.swing.plaf.synth	javax.swing.table	javax.swing.text	javax.swing.text.html
javax.swing.text.html.parser	javax.swing.plaf	javax.swing.text.rtf	javax.swing.tree
javax.swing.undo			

The main package is **javax.swing**.

A SIMPLE SWING APPLICATION

Swing programs differ from both the console-based programs and the AWT-based programs.

It uses two Swing components: **JFrame** and **JLabel**. **JFrame** is the top-level container that is commonly used for Swing applications. **JLabel** is the Swing component that creates a label, which is a component that displays information. The label is Swing's simplest component because it is passive. That is, a label does not respond to user input. It just displays output. The program uses a **JFrame** container to hold an instance of a **JLabel**. The label displays a short text message.

```
// A simple Swing application.
import javax.swing.*;
class SwingDemo {
SwingDemo() {
// Create a new JFrame container.
JFrame jfrm = new JFrame("A Simple Swing Application");
// Give the frame an initial size.
jfrm.setSize(275, 100);
// Terminate the program when the user closes the application.
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// Create a text-based label.
JLabel jlab = new JLabel(" Swing means powerful GUIs.");
// Add the label to the content pane.
jfrm.add(jlab);
// Display the frame.
jfrm.setVisible(true);
}
public static void main(String args[]) {
// Create the frame on the event dispatching thread.
SwingUtilities.invokeLater(new Runnable() {
public void run() {
new SwingDemo();
}
});
}
}
```

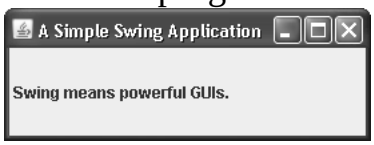
Swing programs are compiled and run in the same way as other Java applications. Thus, to compile this program, you can use this command line:

```
javac SwingDemo.java
```

To run the program, use this command line:

```
java SwingDemo
```

When the program is run, it will produce the window shown in below figure.



It begins by creating a **JFrame**, using this line of code:

```
JFrame jfrm = new JFrame("A Simple Swing Application");
```

This creates a container called **jfrm** that defines a rectangular window with a title bar; close, minimize, maximize, and restore buttons; and a system menu. Next, the window is sized using this statement:

```
jfrm.setSize(275, 100);
```

The **setSize( )** method sets the dimensions of the window, which are specified in pixels. Its general form is shown here:

```
void setSize(int width, int height)
```

In this example, the width of the window is set to 275 and the height is set to 100. By default, when a top-level window is closed (such as when the user clicks the close box), the window is removed from the screen, but the application is not terminated. While this default behaviour is useful in some situations, it is not what is needed for most applications. Instead, you will usually want the entire application to terminate when its top-level window is closed. There are a couple of ways to achieve this. The easiest way is to call **setDefaultCloseOperation( )**, as the program does:

```
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

After this call executes, closing the window causes the entire application to terminate. The general form of **setDefaultCloseOperation( )** is shown here:

```
void setDefaultCloseOperation(int what)
```

The value passed in *what* determines what happens when the window is closed. There are several other options in addition to **JFrame.EXIT_ON_CLOSE**. They are shown here:

```
JFrame.DISPOSE_ON_CLOSE
JFrame.HIDE_ON_CLOSE
JFrame.DO_NOTHING_ON_CLOSE
```

The next line of code creates a Swing **JLabel** component:

```
JLabel jlab = new JLabel(" Swing means powerful GUIs.");
```

JLabel is the simplest and easiest-to-use component because it does not accept user input. It simply displays information, which can consist of text, an icon, or a

combination of the two. The label created by the program contains only text, which is passed to its constructor.
The next line of code adds the label to the content pane of the frame:

```
jfrm.add(jlab);
```

All top-level containers have a content pane in which components are stored. Thus, to add a component to a frame, you must add it to the frame's content pane.
This is accomplished by calling **add( )** on the **JFrame** reference. The general form of **add( )** is shown here:

```
Component add(Component comp)
```

The **add( )** method is inherited by **JFrame** from the AWT class **Container**. By default, the content pane associated with a **JFrame** uses border layout. The version of **add( )** just shown adds the label to the center location. Other versions of **add( )** enable you to specify one of the border regions. When a component is added to the center, its size is adjusted automatically to fit the size of the center.
The last statement in the **SwingDemo** constructor causes the window to become visible:

```
jfrm.setVisible(true);
```

If its argument is **true**, the window will be displayed. Otherwise, it will be hidden. By default, a **JFrame** is invisible, so **setVisible(true)** must be called to show it.
Inside **main( )**, a **SwingDemo** object is created, which causes the window and the label to be displayed. Notice that the **SwingDemo** constructor is invoked using these lines of code:

```
SwingUtilities.invokeLater(new Runnable() {
public void run() {
new SwingDemo();
}
});
```

This sequence causes a **SwingDemo** object to be created on the *event dispatching thread* rather than on the main thread of the application. In general, Swing programs are event-driven. For example, when a user interacts with a component, an event is generated. An event is passed to the application by calling an event handler defined by the application. However, the handler is executed on the event dispatching thread provided by Swing and not on the main thread of the application. Thus, although event handlers are defined by your program, they are called on a thread that was not created by your program.
To avoid problems, all Swing GUI components must be created and updated from the event dispatching thread, not the main thread of the application. However, **main( )** is executed on the main thread. Thus, **main( )** cannot directly instantiate a **SwingDemo** object. Instead, it must create a **Runnable** object that executes on the event dispatching thread and have this object create the GUI.

To enable the GUI code to be created on the event dispatching thread, you must use one of two methods that are defined by the **SwingUtilities** class. These methods are **invokeLater( )** and **invokeAndWait( )**. They are shown here:

```
static void invokeLater(Runnable obj)
static void invokeAndWait(Runnable obj)throws InterruptedException,
InvocationTargetException
```

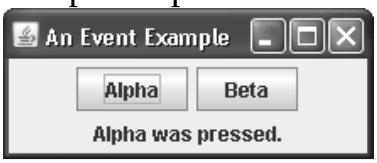
Here, *obj* is a **Runnable** object that will have its **run( )** method called by the event dispatching thread. The difference between the two methods is that **invokeLater( )** returns immediately, but **invokeAndWait( )** waits until **obj.run( )** returns. You can use one of these methods to call a method that constructs the GUI for your Swing application, or whenever you need to modify the state of the GUI from code not executed by the event dispatching thread.

EVENT HANDLING

The preceding example left out one important part: event handling. Because **JLabel** does not take input from the user, it does not generate events, so no event handling was needed. However, the other Swing components *do* respond to user input and the events generated by those interactions need to be handled.

The event handling mechanism used by Swing is the same as that used by the AWT. This approach is called the *delegation event model* and these events are packaged in **java.awt.event**. Events specific to Swing are stored in **javax.swing.event**.

The following program handles the event generated by a Swing push button. Sample output is shown in Figure




```
// Handle an event in a Swing program.
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class EventDemo {
JLabel jlab;
EventDemo() {
// Create a new JFrame container.
JFrame jfrm = new JFrame("An Event Example");
// Specify FlowLayout for the layout manager.
jfrm.setLayout(new FlowLayout());
// Give the frame an initial size.
jfrm.setSize(220, 90);
// Terminate the program when the user closes the application.
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
// Make two buttons.
JButton jbtnAlpha = new JButton("Alpha");
JButton jbtnBeta = new JButton("Beta");
// Add action listener for Alpha.
jbtnAlpha.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
jlab.setText("Alpha was pressed.");
}
});
// Add action listener for Beta.
jbtnBeta.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
jlab.setText("Beta was pressed.");
}
});
// Add the buttons to the content pane.
jfrm.add(jbtnAlpha);
jfrm.add(jbtnBeta);
// Create a text-based label.
jlab = new JLabel("Press a button.");
// Add the label to the content pane.
jfrm.add(jlab);
// Display the frame.
jfrm.setVisible(true);
}
public static void main(String args[]) {
// Create the frame on the event dispatching thread.
SwingUtilities.invokeLater(new Runnable() {
public void run() {
new EventDemo();
}
});
}
}
```

First, notice that the program now imports both the **java.awt** and **java.awt.event** packages. The **java.awt** package is needed because it contains the **FlowLayout** class. The **java.awt.event** package is needed because it defines the **ActionListener** interface and the **ActionEvent** class. The **EventDemo** constructor begins by creating a **JFrame** called **jfrm**. It then sets the layout manager for the content pane of **jfrm** to **FlowLayout**. Notice that **FlowLayout** is assigned using this statement:

```
jfrm.setLayout(new FlowLayout());
```

After setting the size and default close operation, **EventDemo( )** creates two push buttons, as shown here:

```
JButton jbtnAlpha = new JButton("Alpha");
```

```
JButton jbtnBeta = new JButton("Beta");
```
The first button will contain the text “Alpha” and the second will contain the text “Beta.” Swing push buttons are instances of **JButton**. **JButton** supplies several constructors. The one used here is
```
JButton(String msg)
```
The *msg* parameter specifies the string that will be displayed inside the button.
When a push button is pressed, it generates an **ActionEvent**. Thus, **JButton** provides the **addActionListener( )** method, which is used to add an action listener.
The **ActionListener** interface defines only one method: **actionPerformed( )**.
```
void actionPerformed(ActionEvent ae)
```
This method is called when a button is pressed.
Next, event listeners for the button’s action events are added by the code shown here:
```
// Add action listener for Alpha.
jbtnAlpha.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
jlab.setText("Alpha was pressed.");
}
});
// Add action listener for Beta.
jbtnBeta.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
jlab.setText("Beta was pressed.");
}
});
```
Each time a button is pressed, the string displayed in **jlab** is changed to reflect which button was pressed.
Next, the buttons are added to the content pane of **jfrm**:
```
jfrm.add(jbtnAlpha);
jfrm.add(jbtnBeta);
```
Finally, **jlab** is added to the content pane and window is made visible. When you run the program, each time you press a button, a message is displayed in the label that indicates which button was pressed.

Chapter 5
Exploring Swing

Java Jlabel

The object of JLabel class is a component for placing text in a container.
It is used to display a single line of read only text. The text can be changed by an application but a user cannot edit it directly. It inherits JComponent class.

```
public static void main(String args[])
  {
  JFrame f= new JFrame("Label Example");
  JLabel l1,l2;
  l1=new JLabel("First Label.");
  l1.setBounds(50,50, 100,30);
  l2=new JLabel("Second Label.");
  l2.setBounds(50,100, 100,30);
  f.add(l1); f.add(l2);
  f.setSize(300,300);
  f.setLayout(null);
  f.setVisible(true);
  }
  }
```

JTextField

Another commonly used control is JTextField. It enables the user to enter a line of text. JTextField inherits the abstract class JTextComponent, which is the super class of all text components. JTextField defines several constructors. The one we will use is shown here:

```
JTextField(int cols)
```

Here, cols specifies the width of the text field in columns. It is important to understand that you can enter a string that is longer than the number of columns. When you press ENTER when nputting into a text field, an ActionEvent is generated. Therefore, JTextField provides the addActionListener( ) and removeActionListener( )methods. To handle action events, you must implement the actionPerformed( ) method defined by the ActionListener interface. The process is similar to handling action events generated by a button. Like a JButton, a JTextField has an action command string associated with it. By default, the action command is the current content of the text field.

To obtain the string that is currently displayed in the text field, call **getText( )** on the **JTextField** instance. It is declared as shown here:

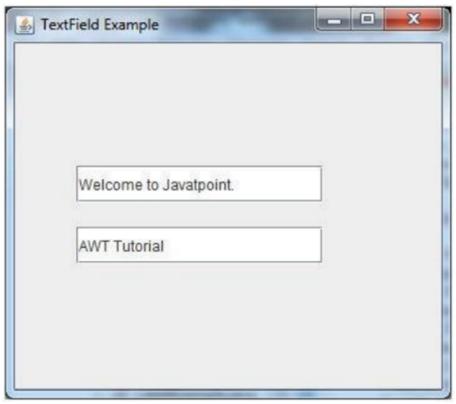
```
String getText( )
```

You can set the text in a JTextField by calling setText( ), shown next:

```
void setText(String text)
```

Here, text is the string that will be put into the text field.

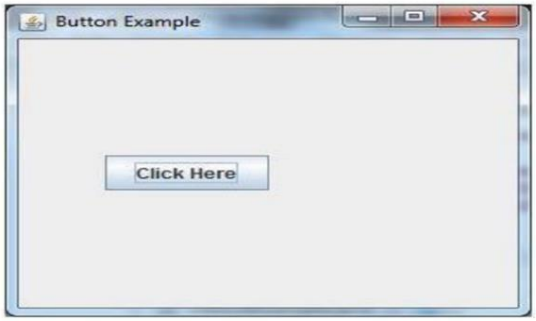
```
import javax.swing.*;
class TextFieldExample
{
public static void main(String args[])
    {
    JFrame f= new JFrame("TextField Example");
    JTextField t1,t2;
    t1=new JTextField("Welcome to Javatpoint.");
    t1.setBounds(50,100, 200,30);
    t2=new JTextField("AWT Tutorial");
    t2.setBounds(50,150, 200,30);
    f.add(t1); f.add(t2);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
    }  }
```



Java JButton

The JButton class is used to create a labelled button. The application result in some action when the button is pushed. It inherits AbstractButton JButton class declaration.

```
import javax.swing.*;
public class ButtonExample {
public static void main(String[] args) {
    JFrame f=new JFrame("Button Example");
    JButton b=new JButton("Click Here");
    b.setBounds(50,100,95,30);
    f.add(b);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true); } }
```



JCheckBox

After the push button, perhaps the next most widely used control is the check box. In Swing, a check box is an object of type JCheckBox. JCheckBox inherits AbstractButton and JToggleButton. Thus, a check box is, essentially, a special type of button. JCheckBox defines several constructors. The one used here is

`JCheckBox(String str)`

It creates a check box that has the text specified by str as a label.

When a check box is selected or deselected (that is, checked or unchecked), an item event is generated. Item events are represented by the ItemEvent class. Item events are handled by classes that implement the ItemListener interface. This interface specifies only one method: itemStateChanged( ), which is shown here:

`void itemStateChanged(ItemEvent ie)`

The item event is received in ie. To obtain a reference to the item that changed, call **getItem( )** on the ItemEvent object.
This method is shown here: `Object getItem( )`
You can obtain the text associated with a check box by calling **getText( )**. You can set the text after a check box is created by calling **setText( )**. These methods work the same as they do for JButton, described earlier. The easiest way to determine the state of a check box is to call the isSelected( ) method. It is shown here:
`boolean isSelected( )`
It returns true if the check box is selected and false otherwise.

```
// Demonstrate JCheckbox.
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="JCheckBoxDemo" width=270 height=50>
</applet>
*/
public class JCheckBoxDemo extends JApplet
implements ItemListener {
JLabel jlab;
public void init() {
try {
SwingUtilities.invokeAndWait(
new Runnable() {
public void run() {
makeGUI();
}
}
);
} catch (Exception exc) {
System.out.println("Can't create because of " + exc);
}
}
private void makeGUI() {
// Change to flow layout.
setLayout(new FlowLayout());
// Add check boxes to the content pane.
JCheckBox cb = new JCheckBox("C");
cb.addItemListener(this);
add(cb);
cb = new JCheckBox("C++");
cb.addItemListener(this);
add(cb);
cb = new JCheckBox("Java");
```

```
cb.addItemListener(this);
add(cb);
cb = new JCheckBox("Perl");
cb.addItemListener(this);
add(cb);
// Create the label and add it to the content pane.
jlab = new JLabel("Select languages");
add(jlab);
}
// Handle item events for the check boxes.
public void itemStateChanged(ItemEvent ie) {
JCheckBox cb = (JCheckBox)ie.getItem();
if(cb.isSelected())
jlab.setText(cb.getText() + " is selected");
else
jlab.setText(cb.getText() + " is cleared");
}
}
```

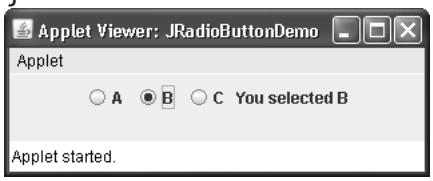
Radio Buttons

Radio buttons are a group of mutually exclusive buttons, in which only one button can be selected at any one time. They are supported by the JRadioButton class, which extends JToggleButton. JRadioButton provides several constructors. The one used in the example is shown here:

```
JRadioButton(String str)
```

```
// Demonstrate JRadioButton
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="JRadioButtonDemo" width=300 height=50>
</applet>
*/
public class JRadioButtonDemo extends JApplet
implements ActionListener {
JLabel jlab;
public void init() {
try {
SwingUtilities.invokeAndWait(
new Runnable() {
public void run() {
```

```
makeGUI();
}
}
);
} catch (Exception exc) {
System.out.println("Can't create because of " + exc);
}
}
private void makeGUI() {
// Change to flow layout.
setLayout(new FlowLayout());
// Create radio buttons and add them to content pane.
JRadioButton b1 = new JRadioButton("A");
b1.addActionListener(this);
add(b1);
JRadioButton b2 = new JRadioButton("B");
b2.addActionListener(this);
add(b2);
JRadioButton b3 = new JRadioButton("C");
b3.addActionListener(this);
add(b3);
// Define a button group.
ButtonGroup bg = new ButtonGroup();
bg.add(b1);
bg.add(b2);
bg.add(b3);
// Create a label and add it to the content pane.
jlab = new JLabel("Select One");
add(jlab);
}
// Handle button selection.
public void actionPerformed(ActionEvent ae) {
jlab.setText("You selected " + ae.getActionCommand());
}
}
```



Java JList

The object of JList class represents a list of text items. The list of text items can be set up so that the user can choose either one item or multiple items.

It inherits JComponent class. **JList** provides several constructors. The one used here is

`JList(Object[ ] items)`

This creates a **JList** that contains the items in the array specified by *items*.

JList is based on two models. The first is **ListModel**. This interface defines how access to the list data is achieved. The second model is the **ListSelectionModel** interface, which defines methods that determine what list item or items are selected. Although a **JList** will work properly by itself, most of the time you will wrap a **JList** inside a **JScrollPane**. This way, long lists will automatically be scrollable, which simplifies GUI design. It also makes it easy to change the number of entries in a list without having to change the size of the **JList** component.

A **JList** generates a **ListSelectionEvent** when the user makes or changes a selection. This event is also generated when the user deselects an item. It is handled by implementing **ListSelectionListener**. This listener specifies only one method, called **valueChanged( )**, which is shown here:

`void valueChanged(ListSelectionEvent le)`

Here, *le* is a reference to the object that generated the event. Both **ListSelectionEvent** and **ListSelectionListener** are packaged in **javax.swing.event**.

By default, a **JList** allows the user to select multiple ranges of items within the list, but you can change this behavior by calling **setSelectionMode( )**, which is defined by **JList**. It is shown here:

`void setSelectionMode(int mode)`

Here, *mode* specifies the selection mode. It must be one of these values defined by **ListSelectionModel**:

`SINGLE_SELECTION`
`SINGLE_INTERVAL_SELECTION`
`MULTIPLE_INTERVAL_SELECTION`

The default, multiple-interval selection, lets the user select multiple ranges of items within a list. With single-interval selection, the user can select one range of items. With single selection, the user can select only a single item.

You can obtain the index of the first item selected, which will also be the index of the only selected item when using single-selection mode, by calling **getSelectedIndex( )**, shown here:

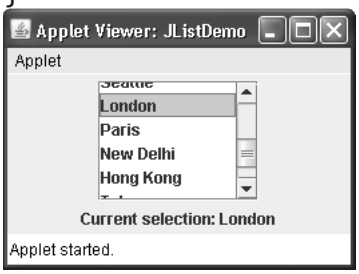
`int getSelectedIndex( )`

Indexing begins at zero. So, if the first item is selected, this method will return 0. If no item is selected, -1 is returned.

```
// Demonstrate JList.
import javax.swing.*;
import javax.swing.event.*;
```

```
import java.awt.*;
import java.awt.event.*;
/*
<applet code="JListDemo" width=200 height=120>
</applet>
*/
public class JListDemo extends JApplet {
JList jlst;
JLabel jlab;
JScrollPane jscrlp;
// Create an array of cities.
String Cities[] = { "New York", "Chicago", "Houston",
"Denver", "Los Angeles", "Seattle",
"London", "Paris", "New Delhi",
"Hong Kong", "Tokyo", "Sydney" };
public void init() {
try {
SwingUtilities.invokeAndWait(
new Runnable() {
public void run() {
makeGUI();
}
}
);
} catch (Exception exc) {
System.out.println("Can't create because of " + exc);
}
}
private void makeGUI() {
// Change to flow layout.
setLayout(new FlowLayout());
// Create a JList.
jlst = new JList(Cities);
// Set the list selection mode to single selection.
jlst.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
// Add the list to a scroll pane.
jscrlp = new JScrollPane(jlst);
// Set the preferred size of the scroll pane.
jscrlp.setPreferredSize(new Dimension(120, 90));
// Make a label that displays the selection.
jlab = new JLabel("Choose a City");
// Add selection listener for the list.
jlst.addListSelectionListener(new ListSelectionListener() {
public void valueChanged(ListSelectionEvent le) {
// Get the index of the changed item.
int idx = jlst.getSelectedIndex();
```

```
// Display selection, if item was selected.
if(idx != -1)
jlab.setText("Current selection: " + Cities[idx]);
else // Otherwise, reprompt.
jlab.setText("Choose a City");
}
});
// Add the list and label to the content pane.
add(jscrlp);
add(jlab);
}
}
```



Java JComboBox

Swing provides a *combo box* (a combination of a text field and a drop-down list) through the **JComboBox** class. A combo box normally displays one entry, but it will also display a drop-down list that allows a user to select a different entry. You can also create a combo box that lets the user enter a selection into the text field.

The **JComboBox** constructor used by the example is shown here:

`JComboBox(Object[ ] items)`

Here, *items* is an array that initializes the combo box. Other constructors are available.

JComboBox uses the **ComboBoxModel**. Mutable combo boxes (those whose entries can be changed) use the **MutableComboBoxModel**.

In addition to passing an array of items to be displayed in the drop-down list, items can be dynamically added to the list of choices via the **addItem( )** method, shown here:

`void addItem(Object obj)`

Here, *obj* is the object to be added to the combo box. This method must be used only with mutable combo boxes.

JComboBox generates an action event when the user selects an item from the list. **JComboBox** also generates an item event when the state of selection changes, which occurs when an item is selected or deselected. Thus, changing a selection means that two item events will occur: one for the deselected item and another for the selected item.

One way to obtain the item selected in the list is to call **getSelectedItem( )** on the combo box. It is shown here:

```
Object getSelectedItem( )
```

You will need to cast the returned value into the type of object stored in the list.

```
import javax.swing.*;
public class ComboBoxExample {
JFrame f;
ComboBoxExample(){
   f=new JFrame("ComboBox Example");
   String country[]={"India","Aus","U.S.A","England","Newzealand"};
   JComboBox cb=new JComboBox(country);
   cb.setBounds(50, 50,90,20);
   f.add(cb);
   f.setLayout(null);
   f.setSize(400,500);
   f.setVisible(true);
}
public static void main(String[] args) {
   new ComboBoxExample();
}
}
```